Simamba: Evaluating Simpson-Style Discretization for Mamba–3 State Space Language Models

Soumil Baldota, Ansen Shia, and David Zhang
School of Engineering and Applied Sciences
Columbia University
New York, NY, USA
{ssb2234, as8008, dwz2107}@columbia.edu

***Abstract*—State space sequence models offer linear-time alternatives to Transformer attention, but their quality and stability depend strongly on the numerical discretization used to update recurrent state. This project studies whether a Simpson-style update can improve the SISO-style recurrence path in our Mamba–3-inspired repository implementation. We implement Simamba, a mixer that replaces the matched trapezoid state update with a learned width-3 Simpson recurrence, and we build the end-to-end training, evaluation, checkpointing, benchmarking, and Hugging Face/vLLM export pipeline needed to test it. On a single Tesla V100, we train controlled 10M-parameter language models on SlimPajama subsets with fixed validation sampling, no-replacement training sampling, AdamW, cosine learning-rate decay, fp32 parameter storage, and gradient health logging. The final controlled runs are stable, but the current Simpson parameterization does not consistently outperform the baselines: it does not beat Mamba–2, and the matched trapezoid controls remain stronger in the 50M-token ablations. In the long-run 500M+ comparison, Simpson+midpoint reaches best validation loss 4.9178 versus 4.8625 for Mamba–2 and 4.9326 for matched trapezoid, but at different training steps. The systems result is mixed: the original repository benchmark has decode-step latency within about 8% of Mamba–3 but prefill hundreds of times slower, while the current repo adds a forward-only improved prefill kernel and profiling harness that still need end-to-end serving validation. The main contribution is therefore a reproducible negative result and a diagnosis: Simpson's negative lag-2 correction is mathematically plausible but currently harder to optimize than the trapezoid update.**

***Index Terms*—state space models, Mamba, language modeling, discretization, Simpson's rule, vLLM, GPU training**

I. Introduction

Selective state space models (SSMs), especially the Mamba family, replace quadratic attention with linear-time recurrent scans while preserving strong language-modeling quality [1], [2]. The latest Mamba–3 line adds inference-first SSM dynamics and specialized kernels; this project targets the SISO-style recurrence path used in our repository implementation [3]. This exposes a numerical question that Transformers avoid: how should a continuous-time SSM be discretized into a stable, trainable, GPU-efficient recurrence? Thus, we hypothesize that a Simpson-style state update (being a higher-order method than the trapezoidal rule for smooth functions) could better approximate the forcing term in a selective SSM and thereby improve language model performance. We refer to the resulting mixer as Simamba. This variant retains the Mamba–3 SISO interface, projections, rotary state, skip path, and language model wrapper, but replaces the width-2 trapezoid recurrence with a width-3 Simpson-inspired update.

Due to reliability issues with the intended multi-GPU A6000 Slurm environment, the study was carried out on a single V100 instead. This limitation turned the study into a controlled negative result rather than a scaling claim: once gradient instability, checkpointing, validation sampling, and memory pressure were addressed, Simpson-style dynamics trained stably but did not consistently outperform Mamba–2 or the matched Simamba trapezoid baseline. Overall, our results suggests that the learned negative lag-2 Simpson term is the primary optimization bottleneck. This highlights how the reparameterization challenge that must be addressed before scaling up training.

II. Literature Review

A. Selective State Space Models

An SSM layer models a sequence through a recurrent state. Mathematically, in simplified continuous time the dynamics are described as follows:

$$\frac{dh(t)}{dt} = A(t)h(t) + B(t)x(t), \tag{1}$$

$$y(t) = C(t)h(t) + D(t)x(t), \tag{2}$$

where the Mamba family makes the dynamics selective by allowing input-dependent parameters [1]. The following iteration, Mamba–2, reformulates the computation using structured state space duality and a hardware-efficient chunk scan [2]. Finally, the recent Mamba–3 paper submitted in March emphasizes inference-first SSM dynamics; our repository implementation exposes a SISO-style path with token-dependent query/key/value-like streams and recurrent state updates [3].

In the language models used here, the recurrent state can be written as a head-wise matrix H_t driven by a value-key outer product:

$$X_t = v_t k_t^\top, \qquad y_t = H_t q_t + D v_t, \tag{3}$$

where q_t, k_t, and v_t are generated by a learned input projection and rotary state. The discretization determines how X_t and previous X_{t-i} terms enter H_t.

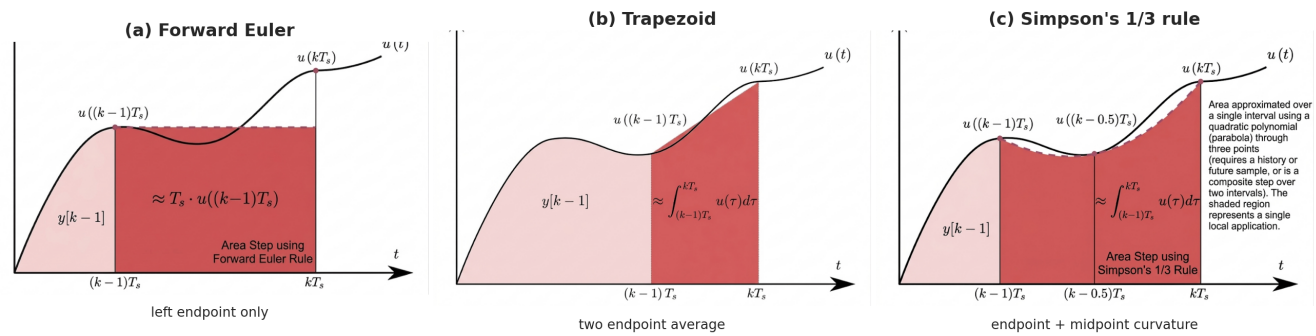


Fig. 1. Discretization intuition from the provided quadrature diagrams. Euler uses only the left endpoint, trapezoid averages the two endpoints, and Simpson's rule adds a midpoint/curvature term. Simamba uses this last idea as a causal learned width-3 SISO update, which introduces the negative lag-2 correction studied in the experiments.

B. Why Simpson's Rule Was Worth Testing

For a scalar linear ODE with a forcing term, the exact discrete update involves an integral over the interval. Trapezoidal integration approximates this integral using only the endpoints, whereas Simpson's rule also incorporates the midpoint, giving a higher-order approximation for smooth forcing functions. Figure 1 illustrates this quadrature intuition. However, in a language-model SSM, the forcing term is a token-dependent value-key product $v_t k_t^\top$, rather than a smooth scalar signal. As a result, the Simpson argument provides motivation but does not guarantee a better recurrence. To remain causal and GPU-efficient, Simamba encodes midpoint and curvature information via a learned, history-only width-3 correction, rather than relying on a noncausal future sample. Our experiments examine whether this numerical intuition holds up under optimization in a learned model.

III. Methodology

A. Architecture

Fig. 2 compares Simamba with the repository Mamba–2 and Mamba–3 block diagrams. The language model embeds token IDs, applies residual RMSNorm mixer blocks, and uses a tied LM head. Simamba keeps the Mamba–3-style projection, RoPE state, skip path, gate, and output projection, but replaces the internal SISO transform with the Simpson recurrence to better capture midpoint and curvature information in the dynamics. Each hidden state is projected into z_t, x_t, B_t, C_t, Δ_t, A_t, discretization controls, and rotary-angle parameters before the recurrence and output path, helping ensure that the model can flexibly adapt its numerical integration while preserving causality and GPU efficiency. Overall, our approach aims to combine the higher-order precision of Simpson-style updates while retaining the established stability of the Mamba–3 baseline.

B. Experimental Scope and Model Setup

The optimization scope is both training and inference: training experiments evaluate whether Simpson discretization improves language-model validation loss, while inference experiments evaluate prefill, decode, memory, kernel traces, and vLLM serving behavior [7]. All controlled language-model experiments use a 10M-parameter decoder-only `MambaLMHeadModel` with 8 layers, $d_{\text{model}} = 160$, sequence length 128, tied embeddings, fp32 residuals/parameters, and a 50,280-token GPT-NeoX vocabulary. For Simamba, each mixer expands to $d_{\text{inner}} = 320$, uses 10 heads of dimension 32, and uses SISO state dimension 64. Training uses micro-batch 8, gradient accumulation to global batch 64, and chunk size 16. Simpson, Simpson + midpoint, and matched trapezoid variants share the same projection and output path; local-conv variants add width-4 causal depthwise mixing over the $x/B/C$ stream.

C. Matched Trapezoid Baseline

The comparison baseline is not stock Mamba–2. Because Mamba–2 incorporates a causal local convolution and a different parameterization, it does not serve as a pure discretization control. To provide a fair comparison, we implemented a matched Simamba trapezoid path using the same projection layout as Simamba. Let

$$\alpha_t = \exp(A_t \Delta_t), \qquad p_t = \sigma(c_t), \tag{4}$$

where c_t is the learned trapezoid coefficient logit. The vectorized training form uses a width-2 contribution:

$$\gamma_t = \Delta_t p_t, \qquad s_t = \gamma_t + \Delta_{t+1}(1 - p_{t+1}), \tag{5}$$

and updates the recurrent state in simplified form as

$$H_t = \alpha_t H_{t-1} + s_t (v_t k_t^\top). \tag{6}$$

This construction ensures that the trapezoid baseline matches Simamba in all architectural aspects except for the internal recurrence rule, so any performance differences can be attributed to the discretization method itself. Rotary state, Q/K biases, D skip, optional z gate, and fixed boundary handling are preserved to maintain consistency, ensuring that the baseline remains a controlled test of whether Simpson-style updates provide a meaningful numerical advantage.

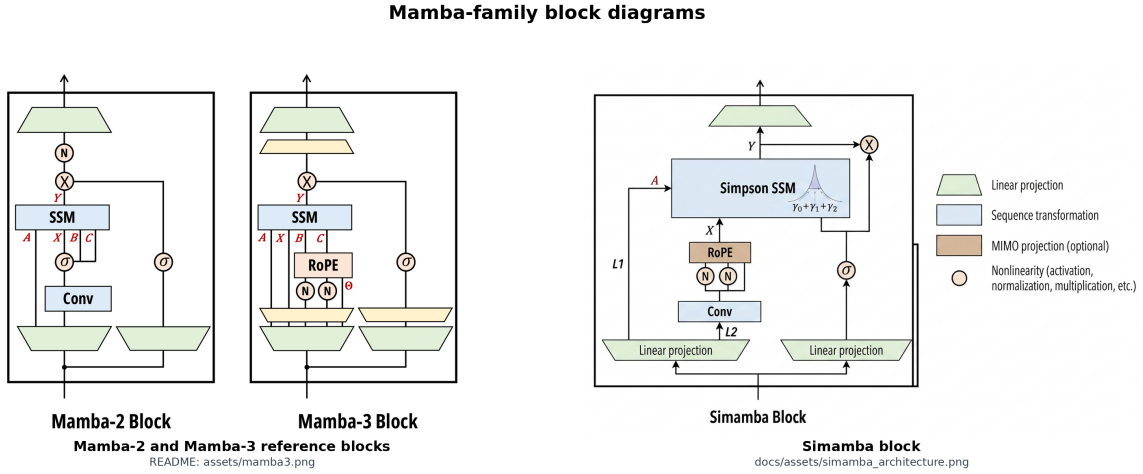


Fig. 2. Architecture comparison using the README Mamba–2/Mamba–3 block diagram and the added Simamba block diagram. Simamba preserves the Mamba–3-style block skeleton but changes the sequence transformation to a Simpson SSM. The matched trapezoid baseline uses the same projections, biases, rotary state, D skip, and output path; only the SISO state-update coefficients change.

D. Simpson-Style Update

Simamba replaces the trapezoid contribution with a learned width-3 Simpson-like recurrence. Define

$$\alpha_t = \exp(A_t \Delta_t), \qquad \alpha_{t,1/2} = \exp(A_t \Delta_t / 2), \tag{7}$$

and a bounded learned Simpson correction

$$s_t = \mathrm{clip}(\sigma(c_t) m_t, 0, 1), \tag{8}$$

where $m_t = 1$ unless midpoint control is enabled, in which case $m_t = \sigma(r_t)$ is learned from an additional projection. The coefficients are

$$\gamma_{0,t} = \frac{\Delta_t}{6}\left(1 + \alpha_{t,1/2}(2 - \frac{1}{2}s_t)\right), \tag{9}$$

$$\gamma_{1,t} = \frac{\Delta_t}{6}\left(\alpha_t + \alpha_{t,1/2}(2 + s_t)\right), \tag{10}$$

$$\gamma_{2,t} = -\frac{\Delta_t}{12}\alpha_{t,1/2} s_t. \tag{11}$$

The state update is

$$\begin{aligned} H_t = \alpha_t H_{t-1} + \gamma_{0,t}(v_t k_t^\top) + \gamma_{1,t}(v_{t-1} k_{t-1}^\top) \\ + \gamma_{2,t}(v_{t-2} k_{t-2}^\top). \end{aligned} \tag{12}$$

The negative lag-2 term $\gamma_{2,t}$ is the key distinction from the trapezoid update. It also appears to be the primary contributor to the noisier gradient norms observed in Appendix Fig. 6. To mitigate potential instability, we introduced two safeguards: a coefficient logit offset that initializes s_t closer to zero, and an optional midpoint control that reduces the effective correction when needed. These controls allow the model to benefit from higher-order Simpson-style updates while maintaining stable training dynamics.

Intuitively, Simpson-style weighting can better capture the curvature of the forcing term over the interval by incorporating information from the midpoint in addition to the endpoints. In the language-model setting, where the forcing term is a token-dependent value-key product rather than a smooth scalar, the learned width-3 recurrence provides a flexible mechanism to approximate this curvature. By encoding history-dependent corrections, Simamba can potentially improve the fidelity of the state update compared to the simpler trapezoid rule, aligning numerical intuition with practical, trainable dynamics.

E. Implementation and Evaluation Protocol

The implementation adds the `Simamba` mixer, reference and Triton Simpson SISO operators, matched trapezoid controls, local-convolution controls, checkpoint/resume metadata, fixed validation sampling, and profiling scripts. The trainable Triton path exposes the Simpson recurrence through a PyTorch autograd wrapper, while a separate forward-only improved prefill kernel is used for systems profiling. The best Simamba and Mamba–2 checkpoints were exported to Hugging Face, with the improved profiling export hosted at https://huggingface.co/kalcy0n/improved-simamba-10m-slimpajama-500m. Additional implementation details are moved to the appendix.

We use SlimPajama [5], streamed from Hugging Face with the GPT-NeoX tokenizer [6], EOS-appended documents, and `uint16` token memory maps. The primary prepared split contains 500M train tokens from 691,532 documents and 50M validation tokens from 49,451 documents; the matched 50M experiments draw budgets from the same prepared data. The final reported runs use one NVIDIA Tesla V100-SXM2 16GB after the intended multi-GPU A6000 Slurm environment proved unreliable. Training uses AdamW [4], linear warmup plus cosine decay, gradient clipping at 1.0, fp32 parameter storage, fixed validation spans, no-replacement epoch sampling, nonfinite-step detection, and W&B logging of configuration, train/validation loss, gradient norms, learning rate, throughput, and memory/profiling summaries.

TABLE I
LONG-RUN 500M+ LANGUAGE-MODEL COMPARISON. SEEN TOKENS ARE ESTIMATED AS STEP COUNT TIMES THE 8192-TOKEN GLOBAL BATCH.

Model	Best val.	Step	Seen	Last val.	Last train
Mamba–2	4.8625	122k	999M	4.8625	4.2041
Simamba Simpson	4.9319	89k	729M	4.9671	4.3852
Simamba Simpson+midpoint	4.9178	71k	582M	4.9889	4.3816
Simamba trapezoid	4.9326	61k	500M	4.9326	4.4823

TABLE II
TAIL TRAINING THROUGHPUT AND GRADIENT-HEALTH SUMMARY FOR THE LONG-RUN MODELS. THROUGHPUT IS MEDIAN TAIL TOKENS/S; GRADIENT NORM IS MEDIAN TAIL PRE-CLIP NORM.

Model	Tok/s	Grad. norm
Mamba–2	16.1k	3.32
Simamba Simpson	12.5k	5.77
Simamba Simpson+midpoint	12.6k	5.16
Simamba trapezoid	16.3k	2.50

IV. EXPERIMENTAL RESULTS

The public W&B dashboard contains the reported runs. For reproducible paper generation without requiring API credentials, the plotting script also reconstructs figures from locally synchronized W&B logs and JSON metric records.

A. Initial Instability

Early runs exhibited unstable gradients before warmup completed; while clipping kept the post-clip updates bounded, it masked large pre-clip norms. Appendix Fig. 4 shows a representative example of this behavior. The controlled runs became stable after switching to fp32 parameters, adding cosine decay, lowering peak learning rates, and logging nonfinite-step/checkpoint metadata.

B. Long-Run 500M+ Token Comparison

The primary comparison starts from one 500M-token pass and then continues the main variants when useful. Table I therefore reports best checkpoints from a long-run 500M+ comparison, not a strict equal-budget comparison. From Table I, we observe that the best validation performance is roughly comparable across variants, with Mamba–2 achieving 4.8625 and the Simpson-style and trapezoid variants slightly higher at 4.9178-4.9326. Notably, the Simpson variants tend to reach their best checkpoints later in terms of seen tokens, reflecting the slower training observed in Table II. Model dimensions are $d_{\text{model}} = 160$, 8 layers, $d_{\text{state}} = 64$, head dimension 32, sequence length 128, and tied input/output embeddings.

The validation curves in Fig. 3 are the strongest evidence against the current Simpson formulation. The corresponding per-run charts are Figs. 16–19. Mamba–2 is the best overall model, but it is not a pure discretization baseline. More importantly, the matched Simamba trapezoid baseline is competitive with and slightly better than default Simpson, while Simpson+midpoint has a better best checkpoint but worse final validation.

TABLE III
50M-TOKEN MATCHED DISCRETIZATION ABLATION.

Variant	Best/final val.	Last train	Skips
Trapezoid	5.8195	5.6111	0
Simpson default	5.9178	5.7161	0
Simpson low-control	5.8929	5.6995	0

Table II highlights two practical observations from the long-run training experiments. First, the Simpson variants (Simamba Simpson and Simpson+midpoint) show reduced training throughput compared to Mamba–2 and the matched trapezoid baseline, achieving roughly 12.5–12.6k tokens/s versus 16k tokens/s. Second, their tail pre-clip gradient norms are noticeably higher, with median values of 5.16–5.77 compared to 2.50–3.32 for the trapezoid and Mamba–2 models. This is consistent with the noisier-gradient behavior illustrated in Appendix Fig. 6 and underscores the gradient sensitivity introduced by the negative lag-2 Simpson term. Together with the earlier observation of early-run instability (Appendix Fig. 4), these results suggest that while Simpson-style updates can be trained stably, they incur higher computational cost and gradient variance relative to simpler discretization schemes.

C. Matched 50M-Token Discretization Ablation

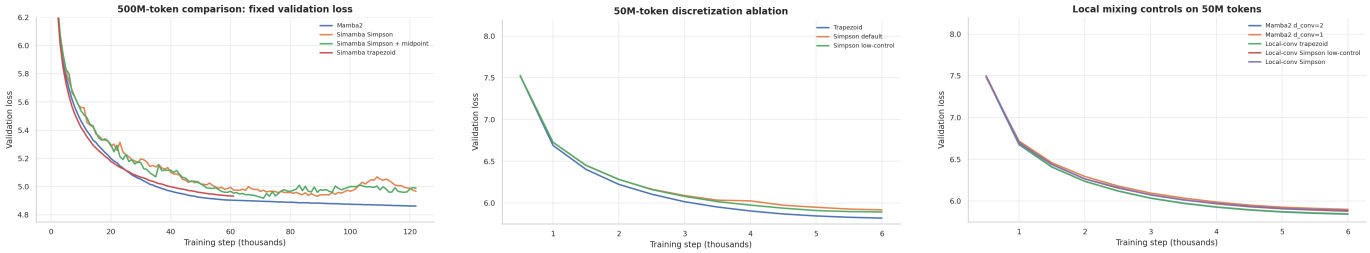
To reduce runtime and isolate the effect of discretization, we ran a 50M-token matched ablation using identical model size, data, batch size, seed, and training schedule across all variants. Table III summarizes the results. The trapezoid baseline achieves the best validation score of 5.8195 and a final training loss of 5.6111, establishing a strong reference. The default Simpson variant lags behind, with a best validation of 5.9178 and a final training loss of 5.7161. Lowering the initial Simpson correction improves performance slightly, yielding a best validation of 5.8929 and final training loss of 5.6995, but it still does not surpass the trapezoid baseline.

These results indicated to us that while reducing the Simpson correction mitigates some of the gradient noise, the width-3 recurrence alone does not provide a consistent numerical advantage at this scale. The corresponding validation curves in Fig. 3 illustrate these trends across training, and the individual run charts (Figs. 20–22) further detail the per-step dynamics for each variant.

D. Local-Convolution Confound

Mamba–2 includes a depthwise causal convolution over the $x/B/C$ stream before the SSM update. We initially had Simamba omit this component. This design choice proves important: as shown in Table IV, reducing or removing the convolution in Mamba–2 degrades quality, with best validation scores rising from 5.8425 for $d_{\text{conv}} = 2$ to 5.9001 for $d_{\text{conv}} = 1$. To ensure a fair comparison and maintain similar local-mixing capacity, we added `--simamba-d-conv 4` to Simamba.

With this adjustment, we can see in Table IV the trapezoid variant of Simamba achieves a best validation of 5.8469, nearly



500M-token comparison | 50M discretization ablation | Local-mixing control

Fig. 3. Main validation-loss evidence. Across the long run, matched ablation, and local-convolution control, Simpson-style updates train stably but do not consistently outperform Mamba–2 or the matched trapezoid controls.

TABLE IV
LOCAL-MIXING CONTROLS ON 50M TOKENS.

Variant	Best/final val.	Last train
Mamba–2 $d_{\text{conv}} = 2$	5.8425	5.6268
Mamba–2 $d_{\text{conv}} = 1$	5.9001	5.6958
Simamba local-conv trapezoid	5.8469	5.6309
Simamba local-conv Simpson low-control	5.8793	5.6701
Simamba local-conv Simpson	5.8890	5.6759

TABLE V
SISO SYSTEMS EVIDENCE SUMMARY. REPOSITORY BENCHMARK ROWS USE B4/P1024/G256. NSYS ROWS USE THE B2/S256/H32/D64 KERNEL TRACE.

Measurement	Mamba–3	Simamba	Impr. Simamba	Takeaway
Repo prefill ms	0.8602	353.3066	–	orig. slow
Repo decode ms/tok	0.0160	0.0173	–	close decode
NSYS region ms	1.883	8.457	0.937	improved SISO
NSYS GPU ms	0.811	7.034	0.453	lower kernel work

matching Mamba–2 with $d_{\text{conv}} = 2$. Simpson-style variants with and without reduced control see slightly higher validation values (5.8793–5.8890), reflecting the combined effects of the higher-order recurrence and local-convolution control. These results confirm that local feature mixing is a critical component for model quality and that Simamba can leverage it effectively while experimenting with Simpson-style updates.

The overlay in Fig. 3 and the per-run charts in Figs. 23–27 show the same pattern. This experiment is important because it prevents a misleading Mamba–2-vs-Simamba conclusion. Some of Mamba–2's advantage comes from local mixing, not only from its state discretization. After matching that mechanism, Simpson still trails the matched trapezoid update.

E. Throughput, Kernel, and Serving Evidence

Training throughput on the V100 is summarized in Appendix Fig. 8. The non-local Simpson variants are slower than Mamba–2 and trapezoid in the 500M runs, while the local-conv 50M runs are closer but still behind the fastest baselines.

The repository SISO benchmark evaluates prefill and decode separately. In the prefill phase, the original Simamba benchmark path is substantially slower than the Mamba–3 fused-kernel path, with slowdowns of 776.9× for B1/P128/G64 and 410.7× for B4/P1024/G256. In contrast, decode-step latency is much closer, with the original path being only about 1.08× slower. To avoid conflating these results, Table V reports the Mamba–3 reference path, the original Simamba path, and the improved forward-only Simamba SISO prefill kernel separately. The vLLM table below, however, refers to the improved Hugging Face/vLLM export rather than the isolated kernel alone.

The old end-to-end-style SISO benchmark shows that Simamba decode is close to Mamba–3, but unfortunately prefill is not competitive. However, the newer NSYS and NCU kernel profiles show that the improved prefill kernel resolves much of this isolated SISO bottleneck. For the B2/S256/H32/d64 configuration, the improved kernel achieves an NSYS region time of 0.937 ms, compared with 1.883 ms for the profiled Mamba–3 path and 8.457 ms for the original Simamba kernel path. Summed CUDA GPU kernel time follows the same pattern: 0.453 ms for the improved Simamba kernel, 0.811 ms for Mamba–3, and 7.034 ms for the original Simamba implementation. The NCU profile supports the same conclusion. The improved kernel increases the L1/TEX cache hit rate to 86.76% while reducing DRAM throughput to 3.77%, indicating that the dominant bottleneck is no longer the severe memory-stall behavior observed in the initial custom kernel. Detailed profiler counters are provided in Appendix Tables VI and VIII.

The vLLM sweep in Appendix Table VII provides a more conservative evaluation because it measures complete exported models rather than only the isolated SISO scan. The improved Simamba export reduces peak GPU memory usage to 0.48–0.62× that of the exported Mamba–2 model, although throughput remains lower, ranging from 0.38–0.87× of Mamba–2 depending on prompt type and prefix-cache settings. Prefix caching highlights the strongest serving-side improvement. For repeated-prefix prompts, TTFT decreases from 23.40 ms to 19.95 ms for Mamba–2, whereas the improved Simamba export reduces TTFT from 29.05 ms to 7.47 ms. However, the worst-case configuration occurs for normal prompts with prefix caching enabled, where throughput drops to 131.4 tok/s compared with 350.3 tok/s for Mamba–2, while TTFT increases to 137.68 ms. These results suggest that improving the SISO kernel alone is insufficient to optimize end-to-end serving performance. Other components, including projection layers, cache management, batching behavior, and prefix-

cache execution paths, continue to dominate overall system efficiency.

Kernel correctness was checked against PyTorch reference implementations before interpreting these timings. The Mamba–3 kernel is checked against a Mamba–3/trapezoid PyTorch reference, while Simamba and the improved prefill kernel are checked against the Simpson PyTorch reference. The Mamba–3 and Simamba Triton kernels pass forward and backward checks on the small fp32 correctness configuration; the improved prefill kernel passes the forward output check, while its backward row is marked not applicable because the improved kernel is a forward-only prototype. Across passed checks, the worst absolute errors are 3.43×10^{-5} for Mamba–3, 1.53×10^{-5} for Simamba, and 1.91×10^{-6} for improved Simamba. Appendix Fig. 15 summarizes the error scale.

The benchmark plots are provided in Appendix Figs. 9, 10, 11, and 12. Collectively, these results demonstrate that the original prefill gap is a clear systems-level bottleneck rather than a consequence of model quality or validation loss. The improved prefill kernel significantly narrows this gap and validates the feasibility of efficient Simamba SISO execution. But while the improved prefill kernel substantially reduces the isolated SISO bottleneck and is promising, additional optimization at the full model-serving level is still required.

F. Overall Result

The overall conclusion is that the current Simpson-style recurrence is trainable and behaves consistently across settings, but it does not yet provide a clear advantage over strong baselines. The best Simamba Simpson checkpoint, the midpoint-control variant, reaches a validation loss of 4.9178, while the best overall language model remains Mamba–2 at 4.8625. Across both the long-run 500M+ runs and the matched 50M-token ablations, the trapezoid controls remain highly competitive, suggesting that the benefit of higher-order discretization is, at this scale, comparable to simpler update rules rather than decisively better.

From a training perspective, Simpson variants show a distinct optimization profile: they tend to converge more slowly, exhibit higher gradient norms, and require additional stabilization mechanisms such as coefficient initialization offsets and optional midpoint control. These adjustments make training reliable, but they also indicate that the added recurrence complexity changes the optimization landscape in a meaningful way. This suggests that the current parameterization may not be the most efficient way to realize higher-order corrections in practice.

Side diagnostics on synthetic state tracking and compression are included in the appendix; neither changes the main negative result.

V. Discussion

A. Why Simpson Underperformed

The results suggest three interacting causes. First, Simpson's higher-order quadrature rationale assumes smooth forcing terms, but token-level language inputs are discontinuous and highly data dependent. Second, the learned negative lag-2 coefficient in (12) can produce cancellation and noisier gradients, especially early in training before the model has learned stable coefficients. Third, Mamba–2's local convolution is a major architectural confound; once we matched local mixing, Simpson improved but still did not beat trapezoid.

This does not prove that Simpson-style discretization is impossible for SSMs. It shows that the current direct parameterization is not enough. A better design may need a constrained positive decomposition, an annealed correction schedule, or a trust-region style bound on the lag-2 term.

B. Systems Lessons

The project also exposed a systems gap. Even if Simpson quality were better, the original prefill implementation would not be practical in production. The decode step is close to Mamba–3, and the improved prefill kernel makes the isolated SISO scan competitive in the B2/S256/H32/d64 profile. The remaining systems work is end-to-end: fused projection layout, local convolution, recurrence, output projection, batching policy, and vLLM cache behavior all need to be measured together before claiming serving speedup. The V100 also lacks some newer Triton functionality used by Mamba–3 kernels, so a portable fallback path is needed for older GPUs.

C. Limitations and Future Work

The most important next steps are:

- Reparameterize the Simpson correction so the negative lag-2 term cannot dominate early training.
- Anneal the Simpson coefficient from zero after the model has learned a stable trapezoid-like recurrence.
- Build a fused local-conv Simamba kernel and a V100-compatible benchmark fallback.
- Run multi-seed 50M-token comparisons only after a single-seed Simpson variant beats the matched trapezoid baseline.
- Evaluate on longer context lengths, where higher-order state updates may have a better chance to matter.
- Add finetuning-aware int4 and pruning experiments rather than one-shot perturbations.
- Integrate the custom Simamba remote-code path deeper into vLLM if the architecture becomes quality-positive.

VI. Conclusion

We implemented and evaluated Simamba, a Simpson-style SISO discretization for Mamba–3-inspired language models. The work establishes a fully reproducible pipeline on a single V100, including data preprocessing, training and evaluation loops, validation protocols, checkpointing, W&B logging, compression analysis, benchmarking, and model export to serving frameworks.

On the modeling side, the results are mixed but informative. Across controlled experiments, the Simpson recurrence does not consistently outperform Mamba–2 or the matched trapezoid baseline. In the 50M-token ablations, the trapezoid variant

is generally stronger, while in the 500M+ runs the Simpson-midpoint configuration achieves competitive best-checkpoint performance but does not retain a clear advantage at final validation. Overall, the evidence suggests that the benefits of the Simpson-style correction are not yet stable across training budgets or evaluation settings.

Taken together, these findings indicate that Simpson-style updates are a viable but not yet dominant discretization choice in this setting. A promising direction for future work is to treat the Simpson correction as a constrained or annealed modification of a stable trapezoid recurrence, rather than as a full replacement, in order to better control optimization stability while preserving potential higher-order benefits.

Artifact Availability

The public project repository is `github.com/hpmls26/simamba`. The public W&B training dashboard is `wandb.ai/ssb2234-columbia/simamba`; it contains tagged baseline and Simamba runs with model configuration, train/validation loss, gradient norms, learning rate, throughput, GPU utilization where available, GPU memory, hyperparameter/ablation sweep metadata, and run tags separating baseline, Simpson, midpoint, trapezoid, and local-conv variants. Profiling logs use the separate W&B project `wandb.ai/ssb2234-columbia/profiling`. The best checkpoints are available as Hugging Face repositories `soumil1/simamba-midpoint-10m-slimpajama-500m` and `soumil1/mamba2-10m-slimpajama-500m`. The improved profiling export is `kalcy0n/improved-simamba-10m-slimpajama-500m`. The local paper assets are generated by `scripts/generate_hpml_paper_assets.py` and profiling statistics through `profiling/run_all_profiling.py`.

References

[1] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[2] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *Proc. International Conference on Machine Learning*, 2024.

[3] A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu, "Mamba-3: Improved sequence modeling using state space principles," arXiv:2603.15569, 2026.

[4] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations*, 2019.

[5] Cerebras Systems, "SlimPajama: A 627B token cleaned and deduplicated version of RedPajama," Hugging Face dataset, 2023. [Online]. Available: https://huggingface.co/datasets/cerebras/SlimPajama-627B

[6] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach, "GPT-NeoX-20B: An open-source autoregressive language model," arXiv:2204.06745, 2022.

[7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symposium on Operating Systems Principles*, 2023.

APPENDIX A
AI TOOL USE DISCLOSURE

Per the HPML AI Use Policy, this submission discloses AI assistance. The team used ChatGPT/Codex for codebase navigation, LaTeX editing, summarizing repository artifacts, consistency checking against generated figures/tables, and draft-review support on team-written material. Final profiling interpretations, performance reasoning, numerical analysis, and scientific conclusions were validated by the team using the repository, W&B logs, profiler exports, and generated paper assets.

APPENDIX B
RESULT FIGURES AND PER-RUN CHARTS

The main text keeps the result narrative and numerical tables close to the claims. The visual evidence is collected here so the results section is not interrupted by large chart exports. The aggregate figures include the instability trace (Fig. 4), main-run training loss (Fig. 5), gradient norms (Fig. 6), best-loss summary (Fig. 7), throughput summary (Fig. 8), repository SISO benchmark latency, throughput, memory, and ratio plots (Figs. 9, 10, 11, and 12), vLLM serving plot (Fig. 13), compression plot (Fig. 14), and kernel-correctness plot (Fig. 15). The per-run figures below then show the same locally logged training histories as separate labelled charts for each reported run.

A. Aggregate Result Figures

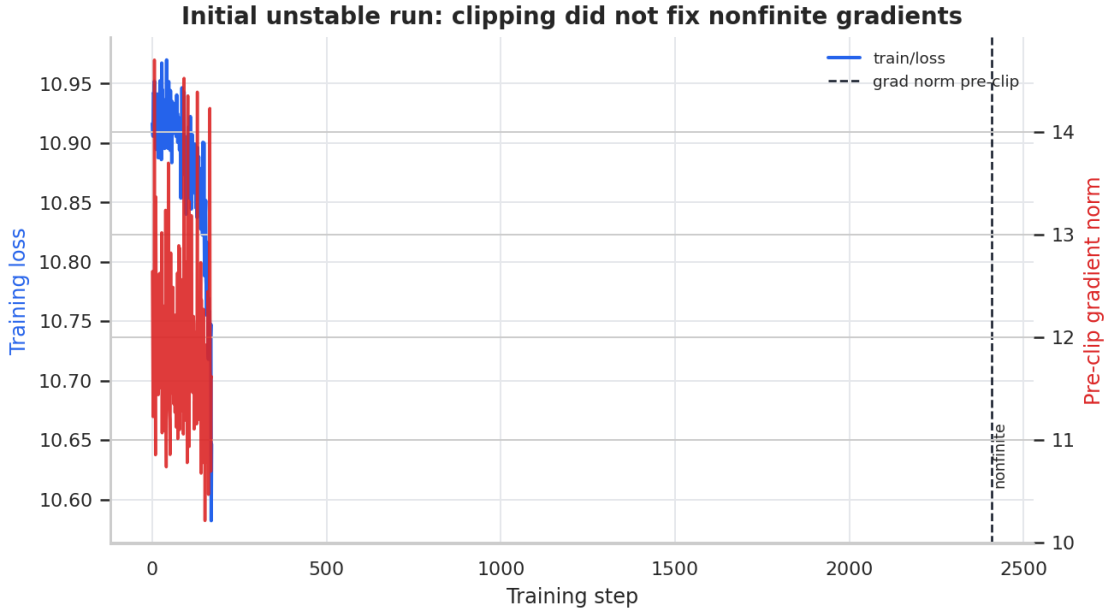


Fig. 4. Initial unstable run. Gradient clipping held the post-clip update at 1.0, but the pre-clip norm stayed large and a nonfinite gradient was detected at step 159.

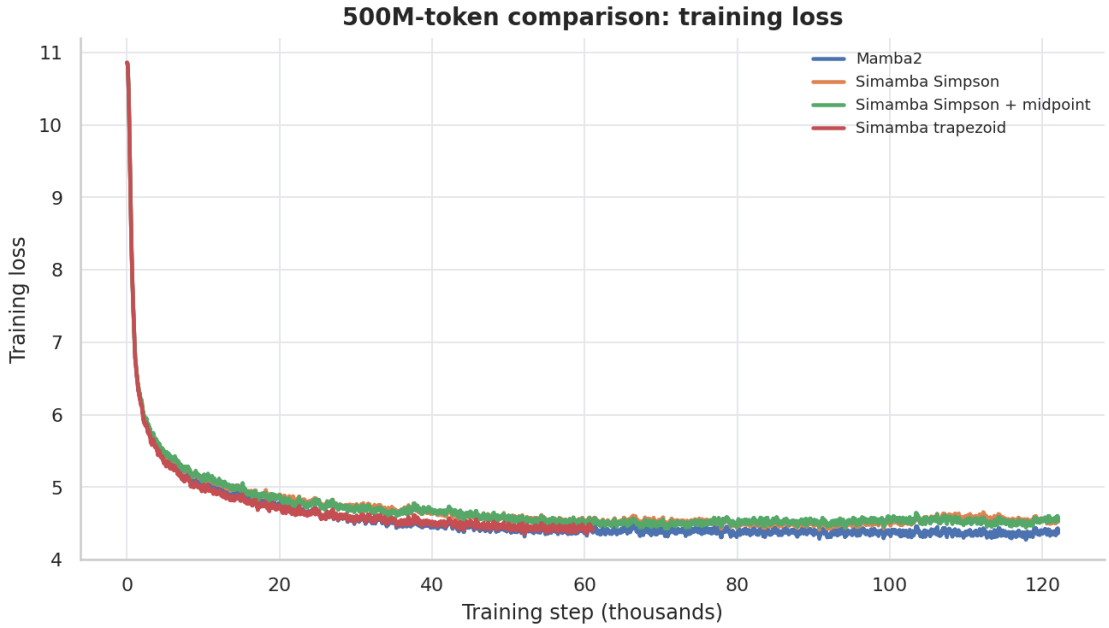


Fig. 5. Smoothed training loss for the 500M-token comparison. All models learn the training distribution, so the validation gap is not caused by a broken data path.

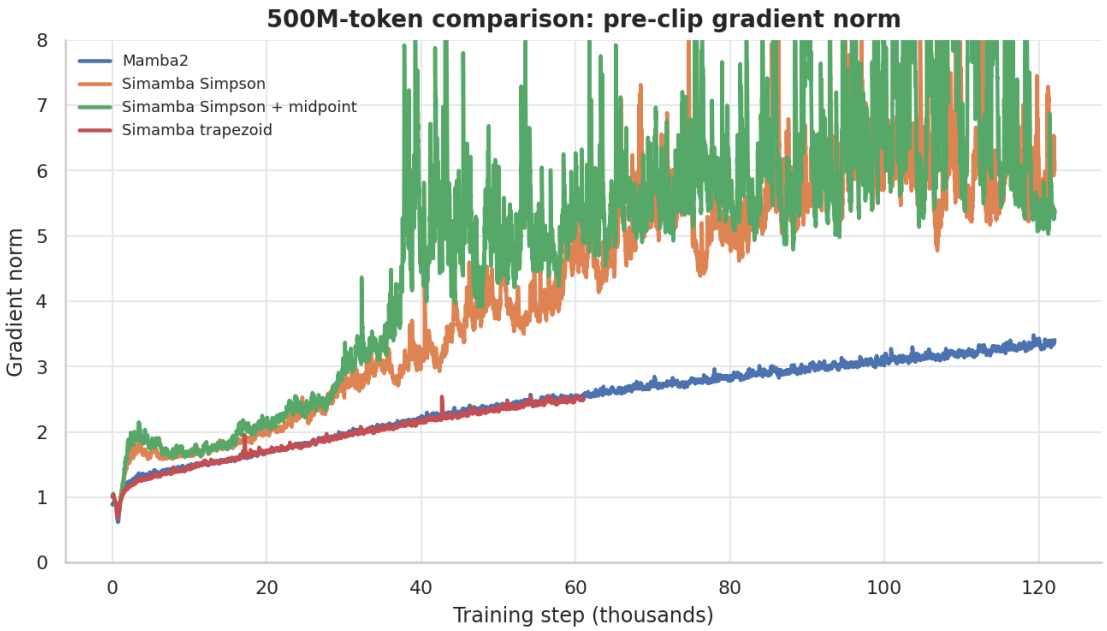


Fig. 6. Smoothed pre-clip gradient norm. The Simpson variants show larger tail gradient norms than Mamba–2 and the matched trapezoid run, consistent with harder optimization.

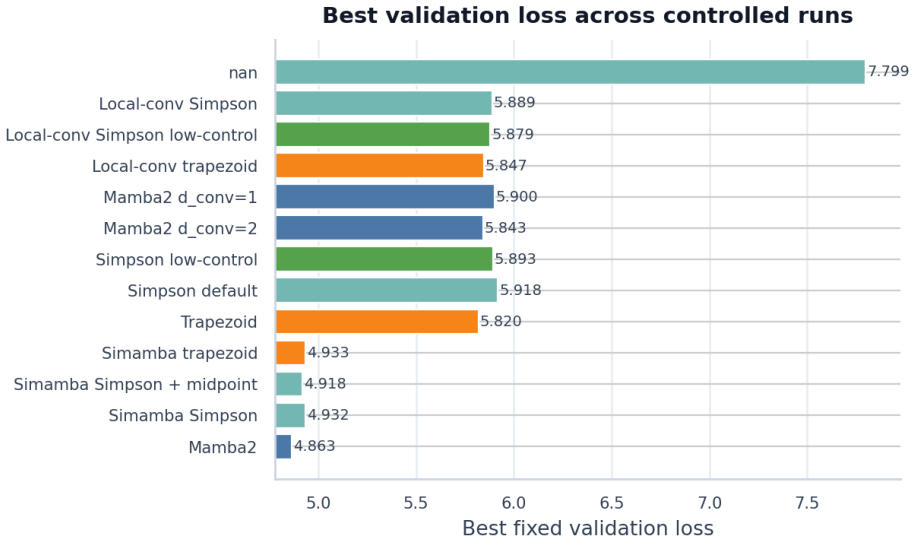


Fig. 7. Best fixed validation loss across the controlled runs. Lower is better.

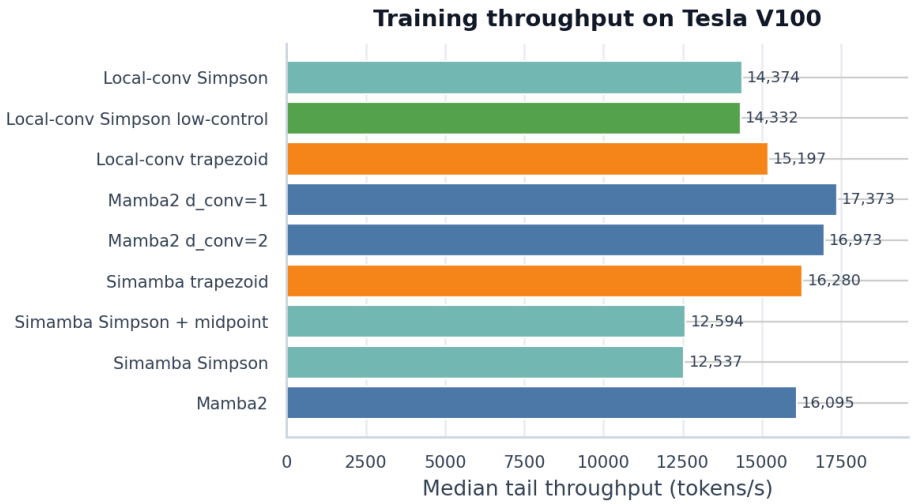


Fig. 8. Median tail training throughput on the V100, computed from the last 100 logged W&B points for each run.

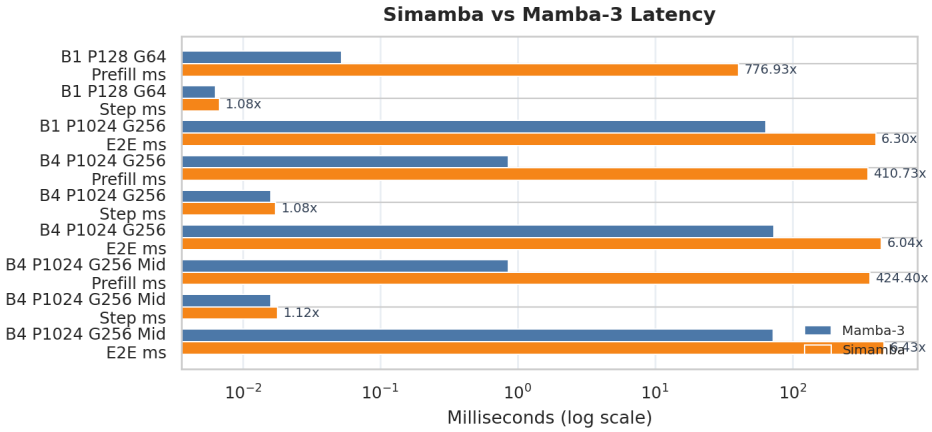


Fig. 9. Repository SISO benchmark latency. Simamba decode overhead is small, but the prefill implementation is not competitive without fused-kernel work.

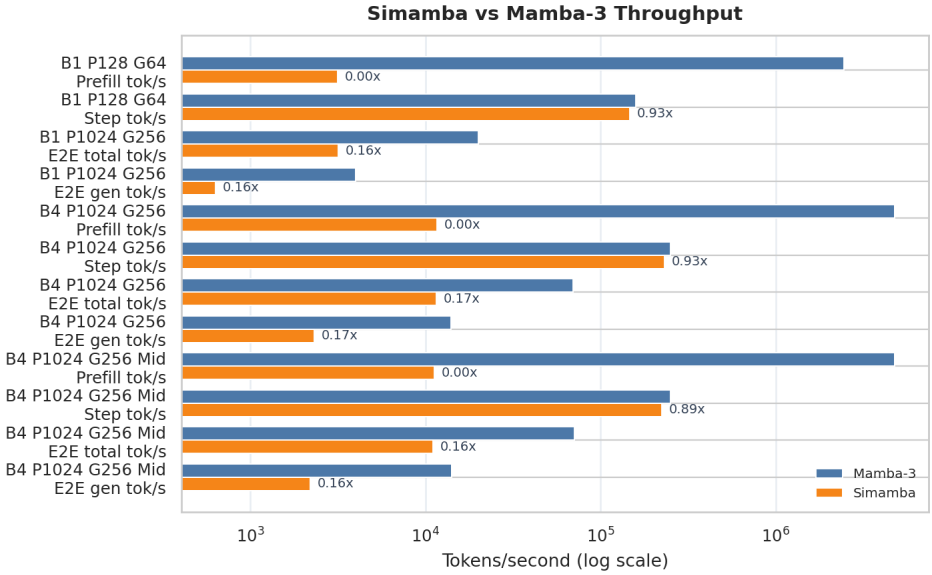


Fig. 10. Repository SISO benchmark throughput. The current prefill gap is a systems bottleneck independent of validation loss.

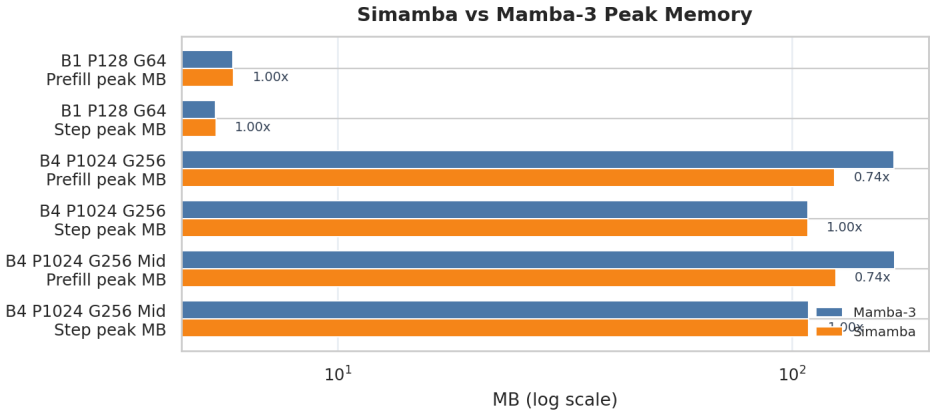


Fig. 11. Repository SISO benchmark memory use. Memory is not the main bottleneck relative to the observed prefill-latency gap.

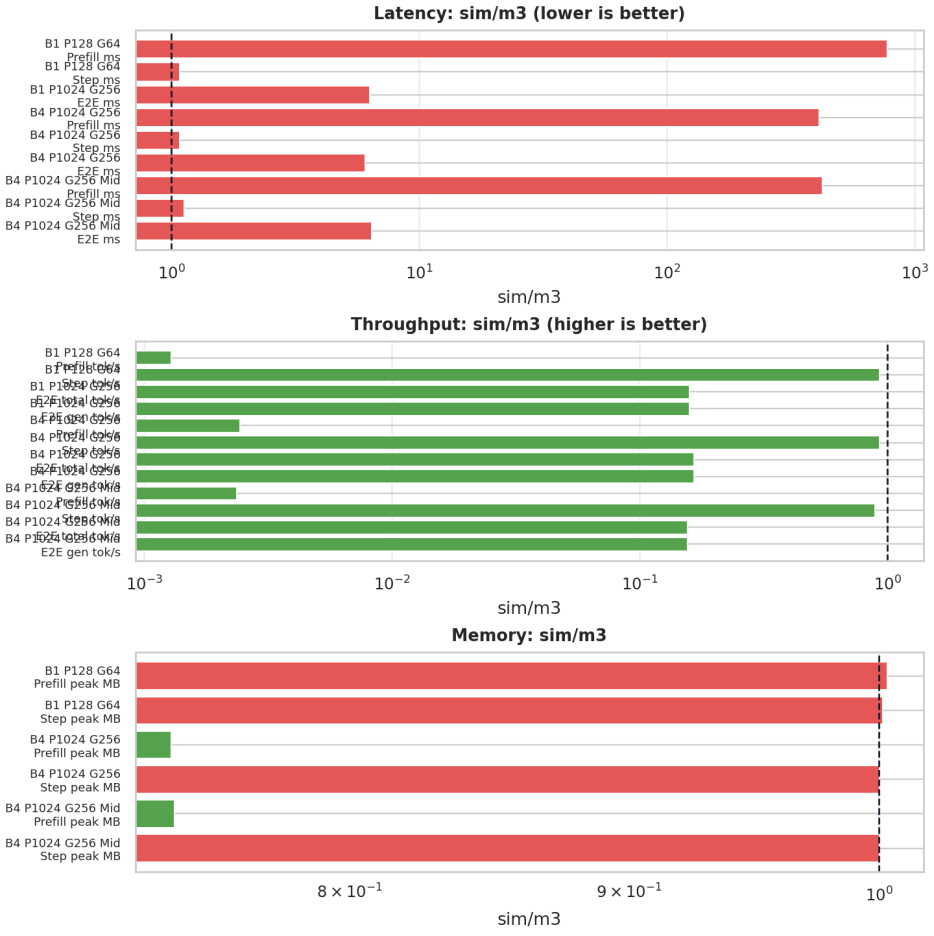


Fig. 12. Repository SISO benchmark ratio summary comparing Simamba with the reference Mamba–3 path.

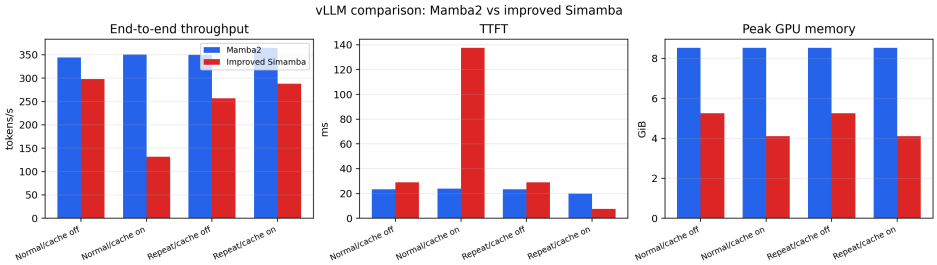


Fig. 13. vLLM serving comparison between the exported Mamba–2 checkpoint and the improved Simamba export. Improved Simamba uses less peak GPU memory but remains slower in overall throughput, with a favorable TTFT result only for repeated-prefix prompts with prefix caching enabled.

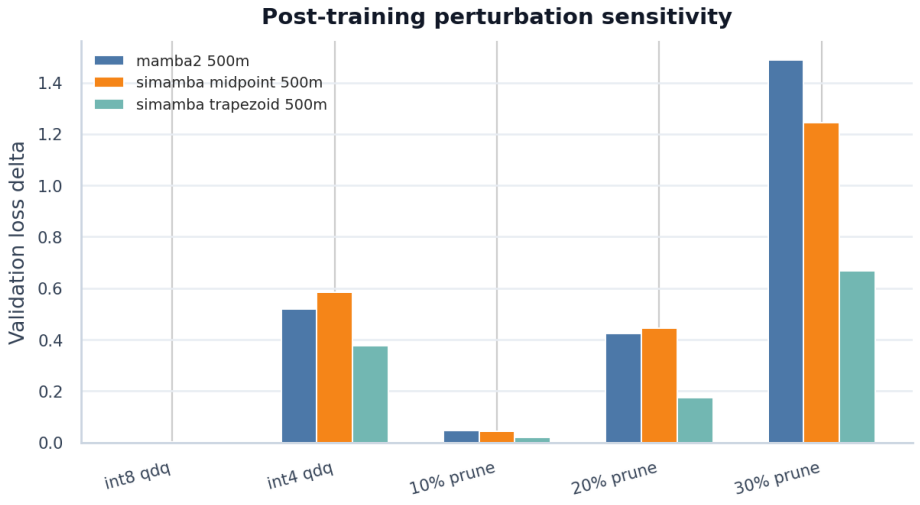


Fig. 14. Validation loss delta after post-training perturbations. Int8 row-wise quantize-dequantize is nearly lossless; int4 and larger pruning ratios require finetuning or better kernels.

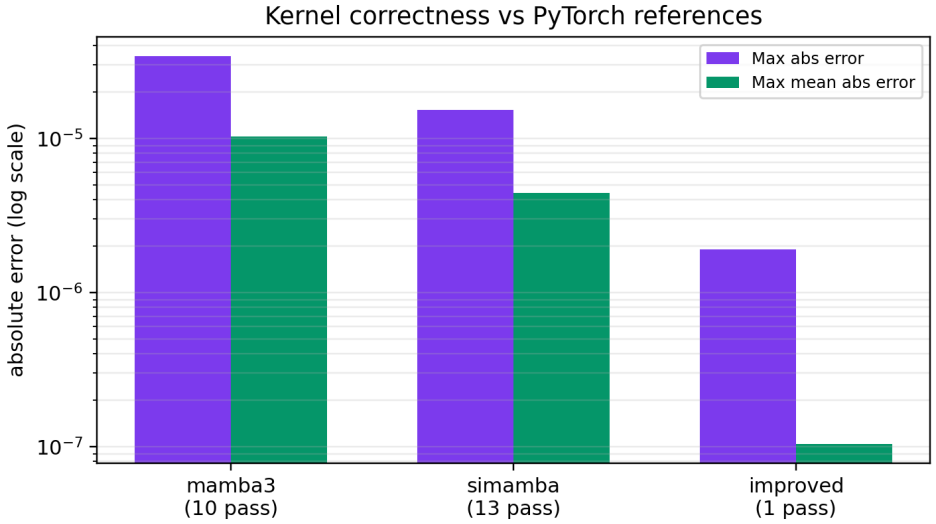


Fig. 15. Kernel correctness error scale of Mamba–3 and Simamba against their respective PyTorch references (Trapezoid and Simpson) for forward and backward checks. The improved Simamba row includes the forward check only because the improved prefill kernel only performs the forward pass, resulting in smaller errors

B. Additional Implementation Details

The main code paths are `simamba.py` for the mixer and inference-cache hooks, `simamba_siso_combined.py` for reference/vectorized Simpson and trapezoid updates, `mamba3_siso_fwd.py` and `mamba3_siso_bwd.py` for Triton kernels, `train_simamba_lm.py` for training/resume/evaluation, `prepare_slimpajama.py` for token memmaps, and `scripts/generate_hpml_paper_assets.py` for reproducible figures. The trainable PyTorch path wraps the Triton SISO recurrence in a custom autograd function, recomputes chunk-start states for the backward pass to reduce activation storage, and keeps a reference path for correctness checks and unsupported edge cases.

The native vLLM fork registers `SimambaForCausalLM`, maps exported fused projection weights into the vLLM module layout, and stores the Simpson recurrent state, rotary-angle state, and key/value history in the Mamba cache. The optimized single-request decode path uses a fused input projection and persistent per-layer recurrent state when prefix caching is disabled; prefix-cache mode uses the conservative cache-all path for correctness. The improved prefill kernel is currently a forward-only profiling artifact, so serving claims in the main text are limited to measured vLLM behavior and isolated SISO-kernel profiles.

C. Kernel Profiling Tables

TABLE VI

NSIGHT SYSTEMS SISO FORWARD TRACE AT $B = 2$, $L = 256$, 32 HEADS, AND HEAD DIMENSION 64 ON THE TESLA V100-SXM2. REGION IS THE PROFILED NVTX RANGE. GPU TIME SUMS CUDA GPU KERNEL DURATIONS INSIDE THE RANGE. SLOWDOWNS ARE RELATIVE TO THE IMPROVED SIMAMBA TRACE.

Path	Region ms	GPU time ms	Kernels	Launch API μs	Sync API μs	Region slow.	GPU slow.
Simamba improved	0.937	0.453	4	66.3	112.7	1.00×	1.00×
Mamba–3 reference	1.883	0.811	2	81.0	686.2	2.01×	1.79×
Simamba original	8.457	7.034	7	122.9	6860.5	9.03×	15.53×

TABLE VII

VLLM COMPARISON BETWEEN THE EXPORTED MAMBA–2 CHECKPOINT AND THE IMPROVED SIMAMBA EXPORT. MEMORY RATIO IS PEAK GPU MEMORY DIVIDED BY MAMBA–2.

Prompt	Cache	Mamba–2 tok/s	Improved tok/s	Mem ratio	Mamba–2 TTFT ms	Improved TTFT ms	Mamba–2 TPOT ms	Improved TPOT ms
Normal	off	344.1	298.1	0.62	23.25	28.93	2.214	2.493
Normal	on	350.3	131.4	0.48	23.96	137.68	2.148	3.382
Repeated	off	350.2	256.8	0.62	23.40	29.05	2.195	2.593
Repeated	on	365.2	288.2	0.48	19.95	7.47	2.169	3.331

TABLE VIII

NSIGHT COMPUTE ISOLATED SISO KERNEL PROFILE FOR THE SAME $B = 2$, $L = 256$, 32-HEAD, HEAD-DIMENSION-64 SETUP. TIME IS INSTRUMENTED PER-LAUNCH KERNEL TIME FROM NCU; IMPROVED SIMAMBA PREFILL USES FOUR CHUNK LAUNCHES FOR THIS SEQUENCE, SO ITS ESTIMATED FULL-SEQUENCE INSTRUMENTED TIME IS ABOUT $4 \times 185.92\ \mu\text{S} = 743.68\ \mu\text{S}$. "MEM" IS GPU COMPUTE-MEMORY THROUGHPUT, "OCC." IS ACHIEVED OCCUPANCY, AND "SMEM" IS DYNAMIC SHARED MEMORY PER BLOCK.

Path	Time	Mem %	DRAM %	L1 hit %	Issue %	Elig. warps/sched.	Occ. %	Regs /thread	smem KB/block
Simamba improved	185.92 μs	30.17	3.77	86.76	30.90	0.44	12.49	128	16.38
Mamba–3 reference	808.48 μs	73.58	73.58	3.86	6.43	0.08	12.50	64	65.54
Simamba original	7.36 ms	57.28	57.28	12.91	2.86	0.03	12.50	32	98.30

D. Side Diagnostics

Synthetic state-tracking tests were added as diagnostics for algorithmic recurrence behavior. The recorded modular-sum Simamba Simpson run did not produce a positive result: final metrics were approximately loss 2.7811 and accuracy 0.0508 at length 128, and loss 2.7805 and accuracy 0.0542 at length 256. These results do not affect the language-model conclusion.

Compression was evaluated with non-destructive one-shot weight perturbations, not finetuning. Row-wise int8 quantize-dequantize is nearly lossless for all checkpoints, while int4 and larger unstructured pruning ratios degrade validation loss substantially. Appendix Fig. 14 visualizes the same deltas.

TABLE IX
COMPRESSION PERTURBATION LOSS DELTAS ON FIXED VALIDATION SPANS.

Checkpoint	int8	int4	10% prune	30% prune
Mamba–2 best	+0.0019	+0.5181	+0.0457	+1.4881
Simamba midpoint	+0.0014	+0.5846	+0.0433	+1.2446
Simamba trapezoid	+0.0013	+0.3771	+0.0207	+0.6680

E. Per-Run Charts

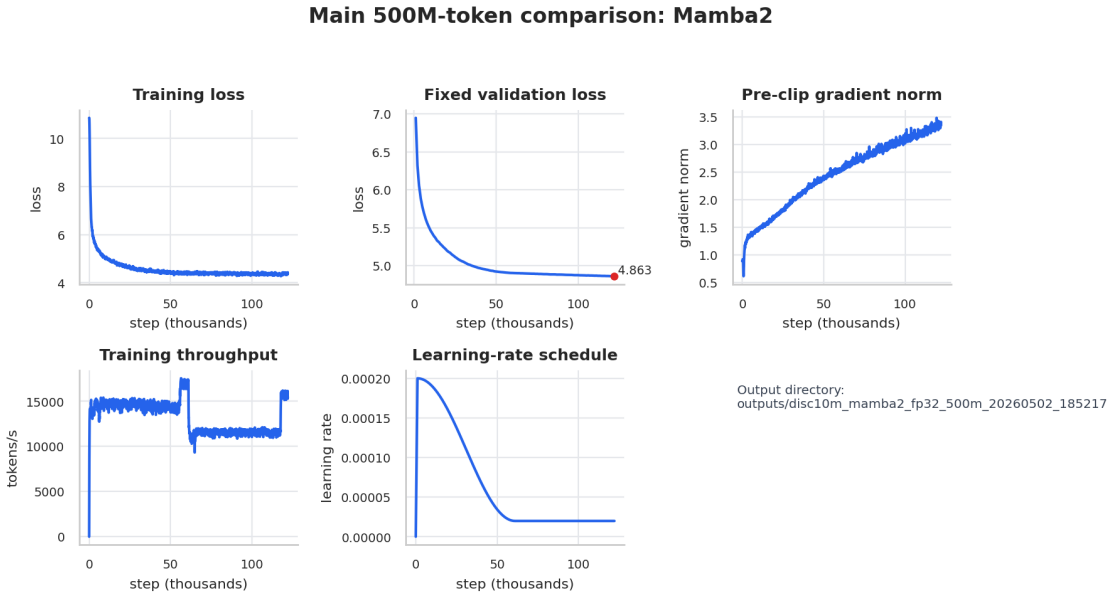


Fig. 16. Per-run chart for Mamba2 in the main 500M-token comparison. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

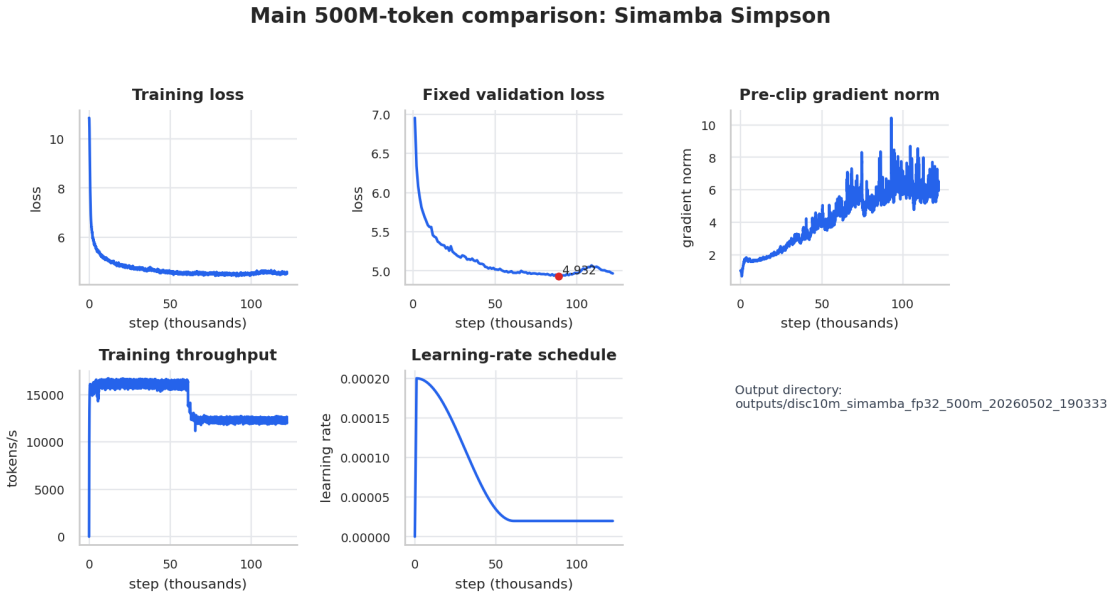


Fig. 17. Per-run chart for Simamba Simpson in the main 500M-token comparison. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

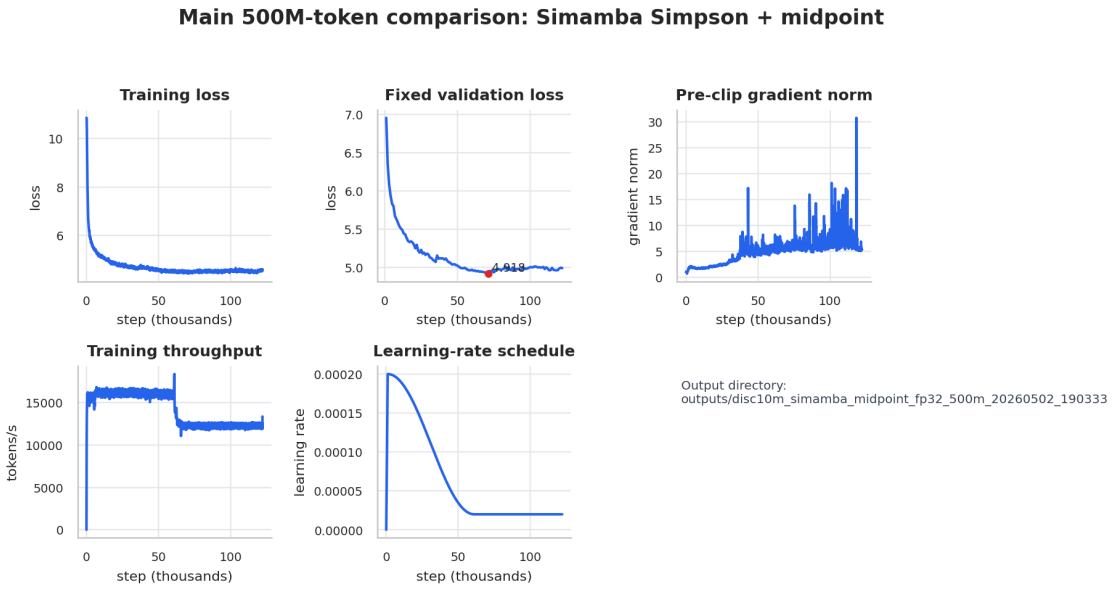


Fig. 18. Per-run chart for Simamba Simpson + midpoint in the main 500M-token comparison. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

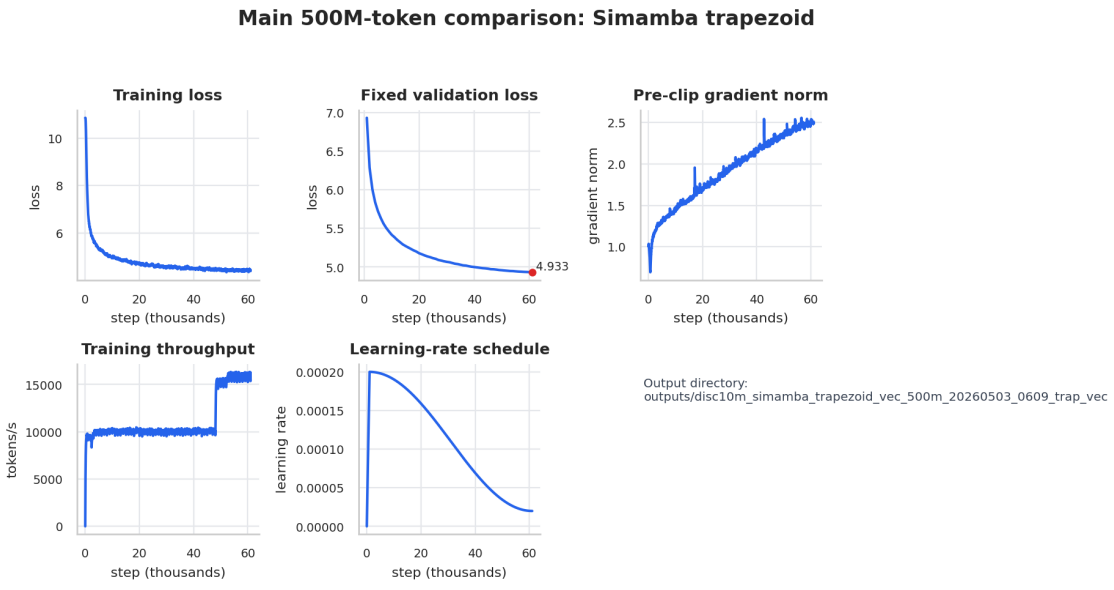


Fig. 19. Per-run chart for Simamba trapezoid in the main 500M-token comparison. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

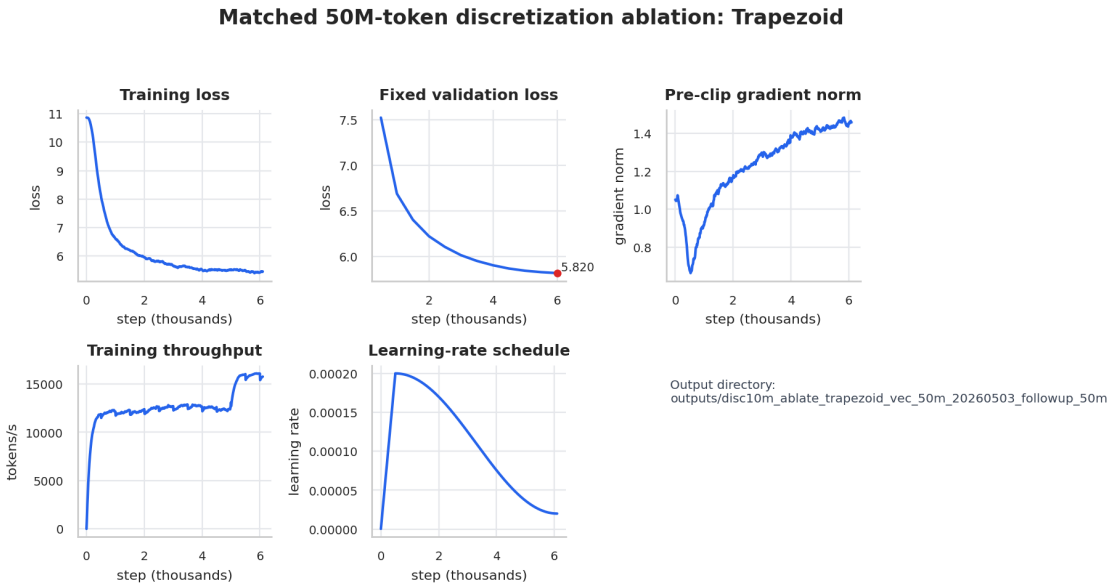


Fig. 20. Per-run chart for Trapezoid in the matched 50M-token discretization ablation. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

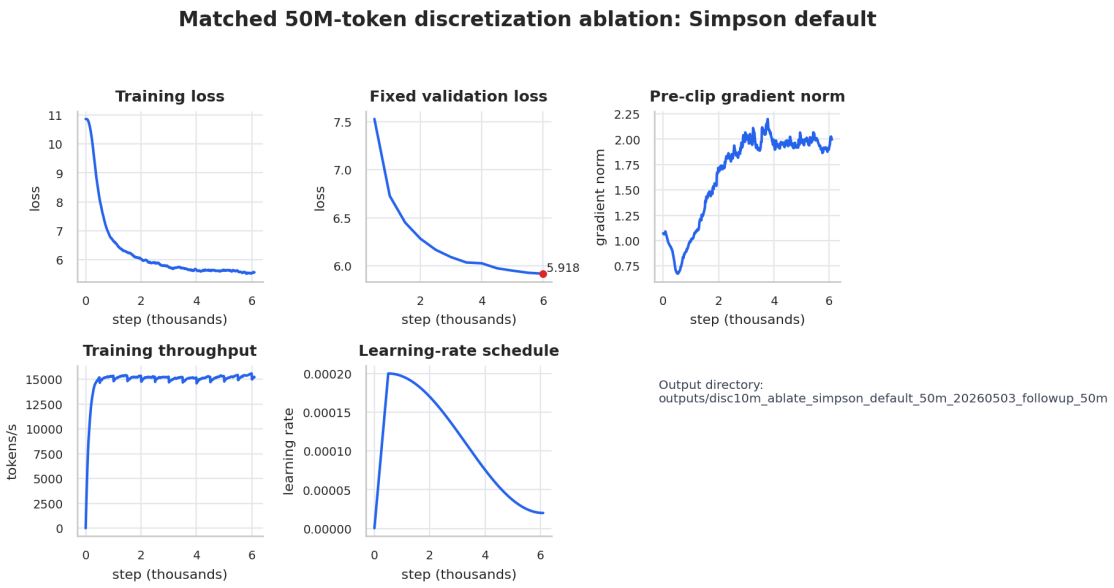


Fig. 21. Per-run chart for Simpson default in the matched 50M-token discretization ablation. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

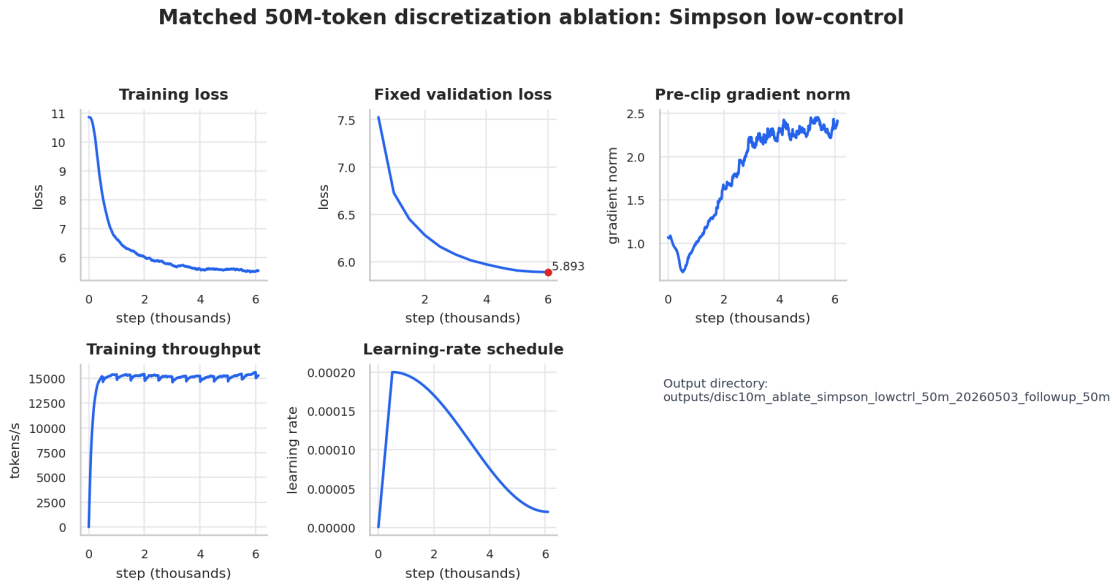


Fig. 22. Per-run chart for Simpson low-control in the matched 50M-token discretization ablation. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

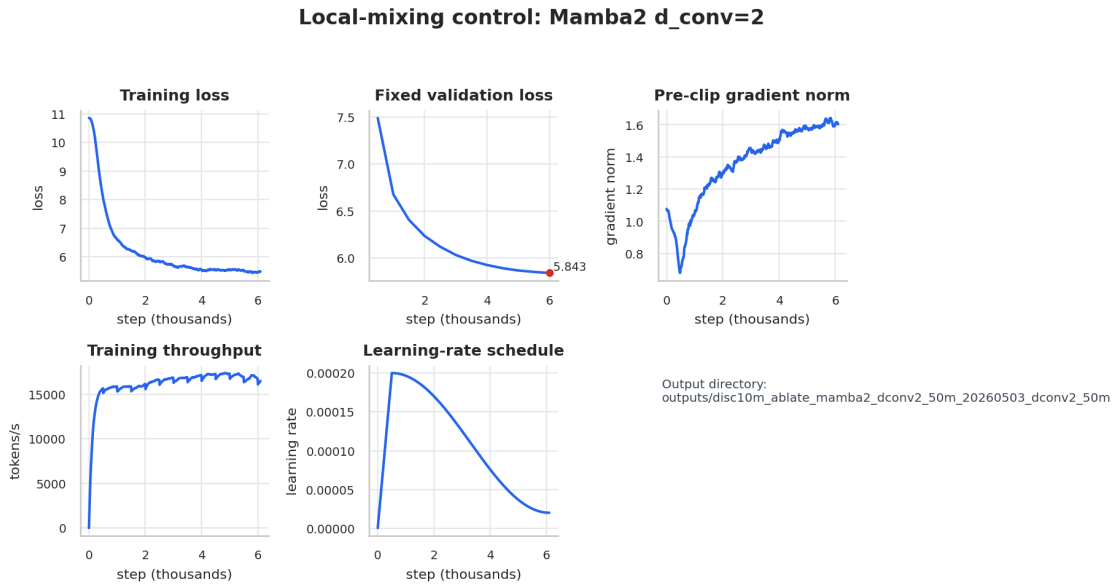


Fig. 23. Per-run chart for Mamba2 d_conv=2 in the local-mixing control. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

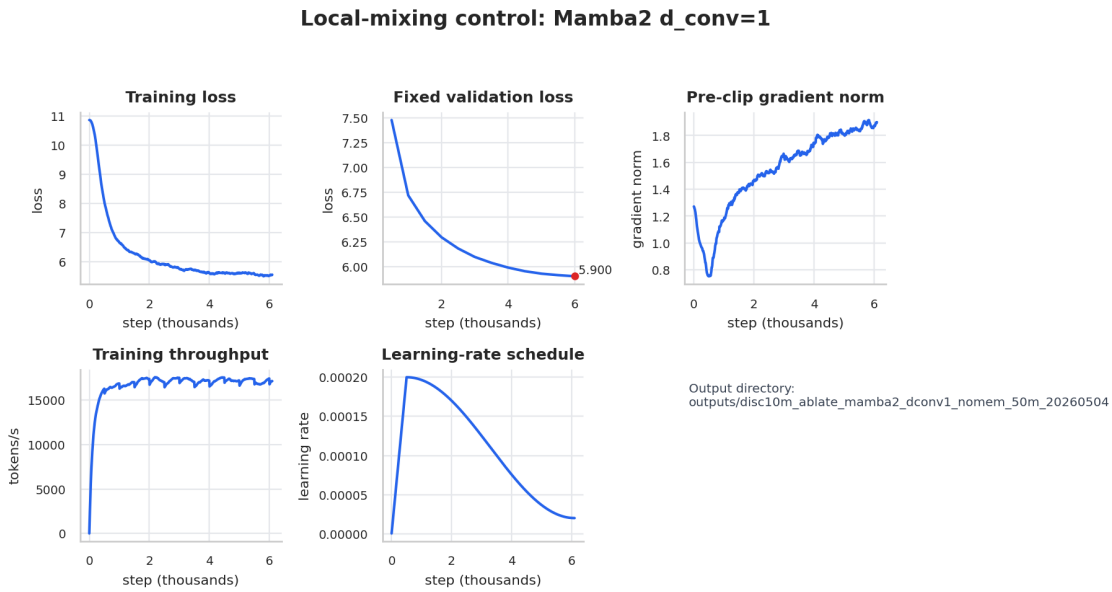


Fig. 24. Per-run chart for Mamba2 d_conv=1 in the local-mixing control. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

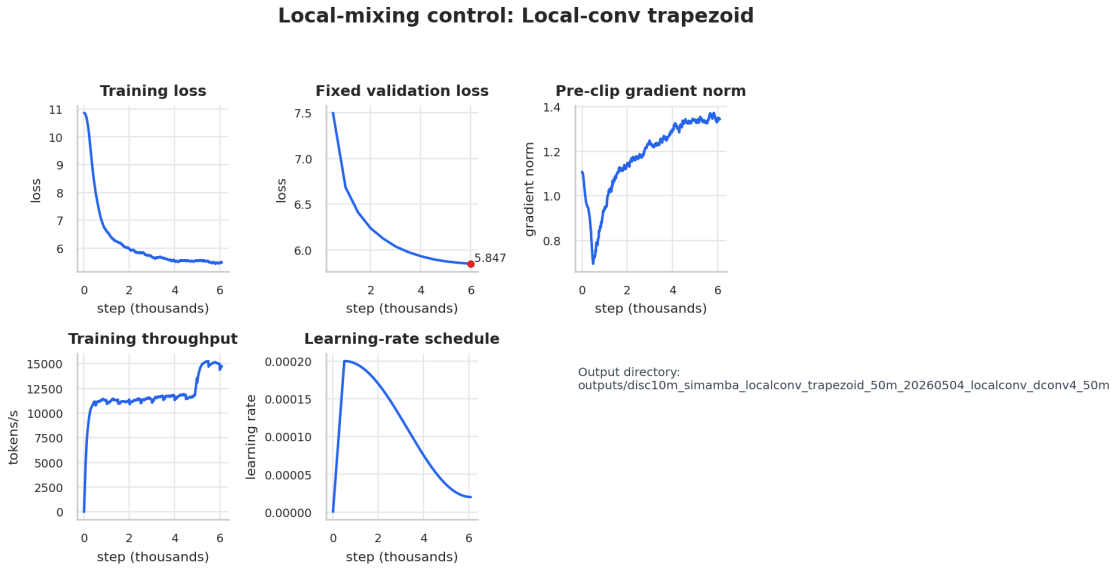


Fig. 25. Per-run chart for Local-conv trapezoid in the local-mixing control. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

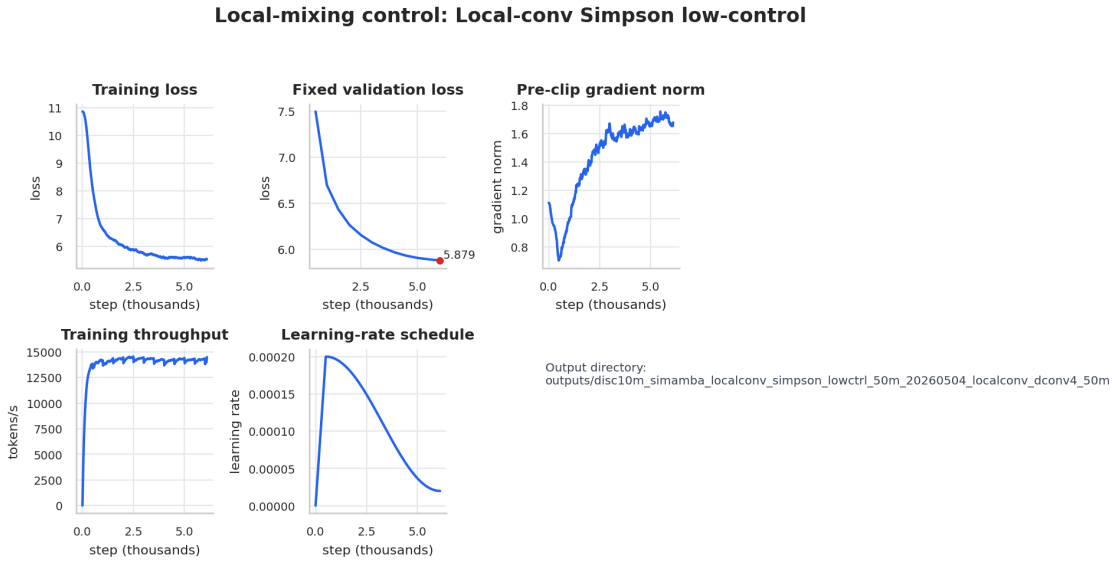


Fig. 26. Per-run chart for Local-conv Simpson low-control in the local-mixing control. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

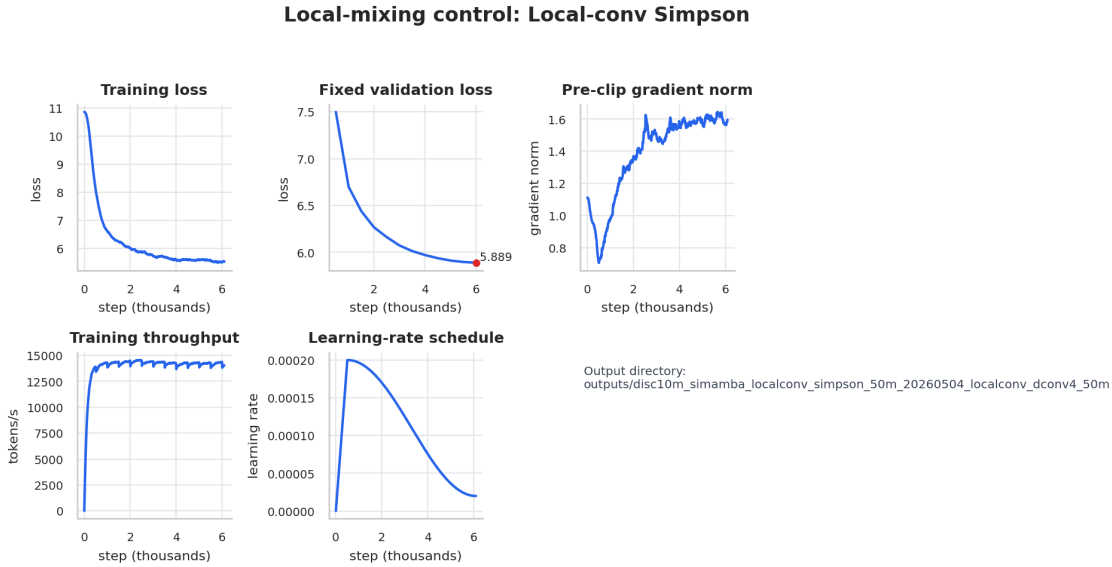


Fig. 27. Per-run chart for Local-conv Simpson in the local-mixing control. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.

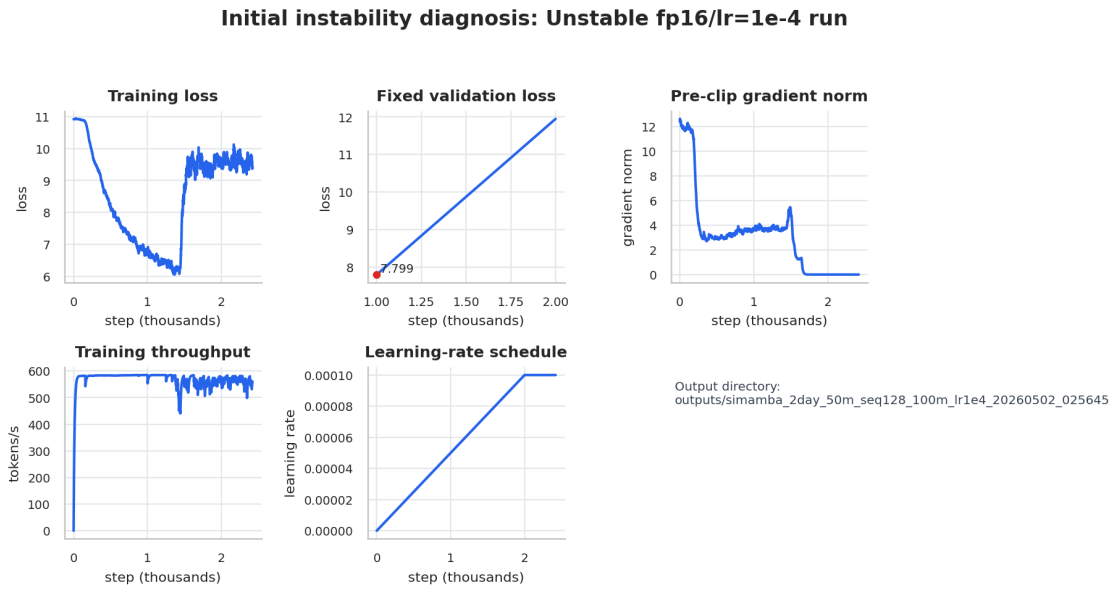


Fig. 28. Per-run chart for Unstable fp16/lr=1e-4 run in the initial instability diagnosis. Panels show training loss, fixed validation loss, pre-clip gradient norm, throughput, and learning-rate schedule when those metrics were logged.